

Worksheet 6: Using the “Generate-Checksum-Hash-SHAs-and-MD5 (Python)” App

This worksheet explains how to open the app in the ZIP you provided and use it to generate file checksums (SHA-1/224/256/384/512, SHA-3 family, and MD5). It also includes verification steps, tips, and troubleshooting.

1) What this app does

- Calculates hash values (unique fingerprints) for any file using:
 - **SHA-1, SHA-224, SHA-256, SHA-384, SHA-512**
 - **SHA-3-224, SHA-3-256, SHA-3-384, SHA-3-512**
 - **MD5**
- Provides a simple **Tkinter** (desktop) GUI where you:
 1. Click **Browse...** to choose a file
 2. Click **Generate** to display all hashes
 3. Click **Reset** to clear the fields

 **Security note:** MD5 and SHA-1 are no longer recommended for secure verification. Prefer **SHA-256** or a SHA-3 variant when integrity matters.

2) What’s inside the ZIP

- `main.py` – launches the GUI
- `checksum.py` – computes the hashes
- `README.md` – quick usage notes
- Image files (screenshots) used by the README

3) Prerequisites

- **Python 3.6+** (SHA-3 is included in Python’s `hashlib` from 3.6 onward)
- **Tkinter** (usually bundled; on some Linux distros you may need to install it)
 - Debian/Ubuntu: `sudo apt install python3-tk`
 - Fedora: `sudo dnf install python3-tkinter`
 - Arch: `sudo pacman -S tk`

To check versions: `python3 --version`

4) Setup (any OS)

1. **Extract** the ZIP to a convenient folder (e.g., Desktop).
2. *(Optional but recommended)* Create and activate a **virtual environment** in that folder:

- macOS/Linux:

```
python3 -m venv .venv
source .venv/bin/activate
```

- Windows (PowerShell):

```
py -3 -m venv .venv
.venv\Scripts\Activate.ps1
```

No third-party packages are required (it only uses the standard library).

5) Run the app

- **macOS/Linux:**

```
python3 main.py
```

- **Windows:**

```
py -3 .\main.py
```

The window titled “**Generate Checksum**” will open.

6) Generate hashes (step-by-step)

1. Click **Browse...** and pick any file (e.g., a PDF, image, installer).
2. Click **Generate**.
3. The app fills in the following fields:
 - SHA-1, SHA-224, SHA-256, SHA-384, SHA-512
 - SHA-3-224, SHA-3-256, SHA-3-384, SHA-3-512
 - MD5
4. Copy any value you need and compare it to a known, trusted hash (e.g., from a download page) to verify integrity.
5. Click **Reset** to clear everything and start again.

7) Verify the results using your OS (optional cross-check)

If you want to confirm a hash outside the app, use one of these commands and compare the output.

Windows (PowerShell)

SHA-256

```
CertUtil -hashfile "C:\path\to\your\file.ext" SHA256
```

MD5

```
CertUtil -hashfile "C:\path\to\your\file.ext" MD5
```

macOS / Linux (Terminal)

SHA-256

```
shasum -a 256 /path/to/your/file.ext
```

MD5

```
md5 /path/to/your/file.ext # macOS
```

```
md5sum /path/to/your/file.ext # Linux
```

8) Understanding the outputs

- Each algorithm produces a **hex string** (0–9, a–f). Different algorithms yield different strings.
- **Identical file → identical hash** (for the same algorithm). If a single bit changes, the hash should change drastically.
- **When to use which:**
 - **SHA-256:** widely supported, strong default for integrity checks
 - **SHA-3-256:** newer design, also strong
 - **MD5/SHA-1:** legacy compatibility only

9) Common issues & fixes

- **Nothing happens when I click Generate**
 - Make sure you selected a file first (the filename row should show a path, not "...")
- **Tkinter not found**
 - Install your platform's Tkinter package (see Prerequisites).
- **Permission denied / can't open file**
 - Try a different file or run the terminal as Administrator (Windows) / use a file in your home folder.
- **Python opens the file in an editor instead of running it**
 - Run from a terminal/shell as shown in **Run the app** above.

10) Stretch activities (optional)

- **Compare algorithms:** Generate SHA-256 and SHA-3-256 for the same file—note both are 64 hex chars (256 bits) yet different values.
- **Tamper test:** Duplicate a file, edit the copy (even one character), then generate hashes for both and compare.
- **Script usage:** In Python, call `checksum.generateHash("/path/to/file")` to receive a dictionary of all hashes.
- **Log to a file:** Paste the results into a text file alongside the filename and date to build a verification log.

11) Checklist (quick assessment)

- ☐ I can start the GUI app
- ☐ I can select a file and generate all hashes
- ☐ I can verify a SHA-256 value using my OS tools
- ☐ I know which algorithms are recommended today
- ☐ I can troubleshoot common setup issues

12) Appendix: Minimal Python snippet

```
from checksum import generateHash
print(generateHash("/path/to/file"))
```

This prints a dictionary like:

```
{
  'sha1': '...', 'sha224': '...', 'sha256': '...', 'sha384': '...', 'sha512': '...',
  'sha3_224': '...', 'sha3_256': '...', 'sha3_384': '...', 'sha3_512': '...',
  'md5': '...'
}
```

Credits & structure

- GUI: `main.py` (Tkinter)
- Hashing logic: `checksum.py` (Python `hashlib`)

That's it—happy verifying!